

Vision–Language Path Planning for Wheeled Mobile Robot

Setup & Execution Instructions

ISC Lab, IIT Kanpur

March 31, 2026

1 Overview

This document describes how to build and run the **VLPathPlanning** ROS workspace on a Clearpath Husky A200 mobile robot equipped with an Intel RealSense depth camera. The system performs *vision–language guided autonomous navigation*: the robot executes a user-defined mission consisting of blind moves and visual object searches using YOLOv8 detection and RedNet semantic segmentation.

1.1 System Architecture

The workspace contains the following ROS packages:

Package	Description
Husky200	Husky A200 driver stack (base, control, bringup)
vehicle_simulator	Gazebo vehicle simulator, RViz config, launch files
local_planner	Obstacle-aware local path planner
terrain_analysis	Terrain traversability analysis (near-field)
terrain_analysis_ext	Extended terrain analysis (far-field)
depth_to_pointcloud	Depth-to-point-cloud conversion, VLNavigation node

Table 1: ROS packages in the workspace.

2 Prerequisites

2.1 Hardware

- Clearpath Husky A200 mobile robot (or equivalent differential-drive platform)
- Intel RealSense D435/D455 depth camera
- NVIDIA GPU (CUDA-capable, for YOLOv8 + RedNet inference)
- Onboard computer running Ubuntu 20.04

2.2 Software

- ROS Noetic (full desktop install recommended)
- Python 3.8+

- **CUDA 11.x+** and **cuDNN** (matching your GPU driver)
- **Gazebo 11** (included with ROS Noetic desktop-full)

2.3 ROS Dependencies

Install the required ROS packages:

```
sudo apt update
sudo apt install -y \
  ros-noetic-pcl-ros \
  ros-noetic-gazebo-ros \
  ros-noetic-realsense2-camera \
  ros-noetic-message-filters \
  ros-noetic-dynamic-reconfigure \
  ros-noetic-tf2-ros \
  ros-noetic-tf2-geometry-msgs \
  ros-noetic-cv-bridge \
  ros-noetic-topic-tools \
  ros-noetic-husky-base \
  ros-noetic-husky-control \
  ros-noetic-husky-bringup \
  ros-noetic-rviz
```

2.4 Python Dependencies

Install the required Python libraries:

```
pip3 install \
  torch torchvision \
  opencv-python \
  numpy \
  open3d \
  Pillow \
  ultralytics
```

Note

Ensure that the PyTorch version matches your CUDA version. Refer to <https://pytorch.org/get-started/locally/> for the correct install command.

3 Building the Workspace

3.1 Step 1: Clone / Copy the Workspace

Place the VLPathPlanning folder in your home directory or any preferred location:

```
# Example: copy to home directory
cp -r /path/to/VLPathPlanning ~/VLPathPlanning
```

```
cd ~/VLPathPlanning
```

3.2 Step 2: Source ROS Environment

```
source /opt/ros/noetic/setup.bash
```

3.3 Step 3: Build with catkin_make

```
cd ~/VLPathPlanning
catkin_make
```

Troubleshooting

If the build fails due to missing dependencies, install them using `rosdep`:

```
rosdep install --from-paths src --ignore-src -r -y
```

3.4 Step 4: Source the Workspace

```
source ~/VLPathPlanning/devel/setup.bash
```

Tip

Add this line to your `~/.bashrc` so it is sourced automatically:

```
echo "source ~/VLPathPlanning/devel/setup.bash" >> ~/.bashrc
```

4 Verify Model Weights

The navigation node requires two pre-trained model files. Ensure they are present at the expected paths:

Model	Expected Path
YOLOv8n	<workspace_root>/yolov8n.pt
RedNet	<workspace_root>/src/depth_to_pointcloud/ src/scripts/model_bestRN1.pth

Table 2: Required model weight files.

Important

If the model paths in `VLNavigation.py` reference a different location (e.g., `/home/administrator/...`), update them to match your system. Specifically, edit:

- Line 37: `CHECKPOINT_PATH` (RedNet model)
- Line 115: YOLO model path
- Line 137: Mission plan JSON path

5 Configuring the Mission

The robot executes missions defined in a JSON file. By default, this file is located at:

```
src/depth_to_pointcloud/src/scripts/mission_plan.json
```

5.1 Mission JSON Format

The mission file is a JSON array of action steps. Two action types are supported:

5.1.1 MOVE – Blind Odometry Move

Moves the robot a fixed distance forward with an optional yaw rotation.

```
{
  "action": "MOVE",
  "desc": "Move forward 0.5 m",
  "forward_dist": 0.5,
  "yaw_delta": 0.0
}
```

Field	Type	Description
<code>action</code>	string	Must be "MOVE"
<code>desc</code>	string	Human-readable description (optional)
<code>forward_dist</code>	float	Distance to travel (metres)
<code>yaw_delta</code>	float	Heading change before moving (radians)

5.1.2 FIND – Visual Object Search

Rotates and scans the environment using YOLOv8 to find a specific object class, then drives towards it using visual servoing.

```
{
  "action": "FIND",
  "target": "chair",
}
```

```

    "id": 56,
    "thresh": 1.3
  }

```

Field	Type	Description
action	string	Must be "FIND"
target	string	Human-readable object name
id	int	COCO class ID for YOLOv8 detection
thresh	float	Approach distance threshold (metres); the robot stops when the median depth to the target falls below this value

5.2 Example Mission

```

[
  {
    "action": "MOVE",
    "desc": "Move a little ahead",
    "forward_dist": 0.5,
    "yaw_delta": 0.0
  },
  {
    "action": "FIND",
    "target": "chair",
    "id": 56,
    "thresh": 1.3
  },
  {
    "action": "FIND",
    "target": "potted plant",
    "id": 58,
    "thresh": 1.7
  },
  {
    "action": "FIND",
    "target": "person",
    "id": 0,
    "thresh": 1.0
  }
]

```

This mission instructs the robot to:

1. Move 0.5 m forward.
2. Scan for a *chair* (COCO ID 56) and approach within 1.3 m.

3. Scan for a *potted plant* (COCO ID 58) and approach within 1.7 m.
4. Scan for a *person* (COCO ID 0) and approach within 1.0 m.

Common COCO Class IDs

0: person 56: chair 58: potted plant 39: bottle 62: TV 73: laptop Full list:
<https://docs.ultralytics.com/datasets/detect/coco/>

6 Updating Hardcoded Paths

Before running, update the following hardcoded paths in the source files to match your system:

6.1 In VLNavigation.py

```
# Line 37 - RedNet checkpoint
CHECKPOINT_PATH = '/home/<your_user>/VLPathPlanning/src/
depth_to_pointcloud/src/scripts/model_bestRN1.pth'
# Line 115 - YOLO model
self.yolo_model = YOLO('/home/<your_user>/VLPathPlanning/yolov8n.pt')
# Line 137 - Mission plan
plan_path = "/home/<your_user>/VLPathPlanning/src/
depth_to_pointcloud/src/scripts/mission_plan.json"
```

6.2 In system_iscLabIITK.launch

```
<!-- Line 23 - Camera ROI YAML (update or comment out) -->
<node pkg="dynamic_reconfigure" type="dynparam"
  name="set_auto_exposure_roi"
  args="load /camera/rgb_camera/auto_exposure_roi
/home/<your_user>/VLPathPlanning/src/vehicle_simulator/
cameraRoi.yaml" />
```

7 Running the System

7.1 Step 1: Launch the Robot System

Open a terminal window and run:

```
source ~/VLPathPlanning/devel/setup.bash
roslaunch vehicle_simulator system_iscLabIITK.launch
```

This launches the following subsystems:

- **Husky base driver** – motor control and odometry
- **RealSense camera** – RGB-D image streams

- **Local planner** – obstacle-aware path planning
- **Terrain analysis** – traversability mapping
- **Vehicle simulator** – state estimation and Z-height adjustment
- **RViz** – 3D visualisation
- **TF publishers** – coordinate frame transforms

Wait for Startup

Allow 5–10 seconds for all nodes to initialise and the RealSense camera to begin streaming before proceeding.

7.2 Step 2: Run the Navigation Node

In a second terminal:

```
source ~/VLPathPlanning/devel/setup.bash
roslaunch depth_to_pointcloud VLNavigation.py
```

Alternatively, run it directly:

```
python3 ~/VLPathPlanning/src/depth_to_pointcloud/\
src/scripts/VLNavigation.py
```

Execution Flow

Upon startup, VLNavigation.py will:

1. Load the RedNet semantic segmentation model onto GPU.
2. Load the YOLOv8 object detection model.
3. Read the mission plan from `mission_plan.json`.
4. Begin executing steps sequentially: **MOVE** steps use odometry; **FIND** steps use 360° scanning and visual servoing.

8 Runtime Behaviour

8.1 State Machine

The navigation node operates as a finite state machine:

8.2 Key Parameters

9 ROS Topics

State	Description
IDLE	No active mission; robot is stationary.
WAIT_FOR_REACH	Executing a MOVE or driving towards a detected target. Goal completion is checked via odometry (for MOVE) or visual depth (for FIND).
SEARCH_SCAN	Robot is stationary, scanning consecutive camera frames for the target object using YOLOv8.
SEARCH_MOVE	Rotating in place by 30° increments to scan new directions.

Table 3: VLNavigation state machine states.

Parameter	Default	Description
SYSTEM_LATENCY	0.15 s	Compensation for sensor-to-publish delay
SCAN_ANGLE_STEP_DEG	30°	Rotation step during object search
FRAMES_TO_SCAN	10	Frames to analyse before rotating
YOLO confidence	0.4	Minimum detection confidence
Detection confirmation	3 frames	Consecutive detections required to lock target

Table 4: Tunable parameters in VLNavigation.py.

Topic	Type	Description
/camera/depth/image_rect_raw	sensor_msgs/Image	Aligned depth image from RealSense
/camera/color/image_raw	sensor_msgs/Image	RGB image from RealSense
/state_estimation	nav_msgs/Odometry	Robot pose estimate
/velodyne_points	sensor_msgs/PointCloud2	Generated obstacle point cloud (published by VLNavigation)
/cmd_vel	geometry_msgs/Twist	Velocity commands
/wpts	geometry_msgs/PointStamped	Waypoint goals for local planner
/mission_marker	visualization_msgs/Marker	RViz goal marker

Table 5: Key ROS topics used by the system.

10 Quick-Start Summary

Complete Workflow

1. Install prerequisites (ROS Noetic, Python packages, CUDA).
2. Build the workspace:

```
cd ~/VLPathPlanning && catkin8make
source devel/setup.bash
```


11 Troubleshooting

ModuleNotFoundError: No module named 'ultralytics'

Install the YOLO package: `pip3 install ultralytics`

CUDA out of memory

Reduce camera resolution in the launch file or use a smaller YOLO model (e.g., `yolov8n.pt`).

Robot does not move

Ensure `/state_estimation` topic is being published. Check the Husky base driver logs.

Target not found (infinite spinning)

Verify the correct COCO class ID in `mission_plan.json`. Check YOLO detection confidence threshold.

TF lookup errors

Ensure the static transform publishers in the launch file are running. Check with `roslaunch tf tf_echo map sensor`.

Point cloud not visible in RViz

Verify the `/velodyne_points` topic is being published and the fixed frame in RViz is set to `map`.